



LeetCode 215: Kth Largest Element in an Array

1. Problem Title & Link

Title: LeetCode 215: Kth Largest Element in an Array

Link: <https://leetcode.com/problems/kth-largest-element-in-an-array/>

2. Problem Statement (Short Summary)

Given an integer array:

nums

and an integer:

k

Return the:

kth largest element

in the array.

Note:

Not kth distinct element.

3. Examples (Input → Output)

Example 1

Input:

nums = [3,2,1,5,6,4]

k = 2

Output:

5

Sorted:

[1,2,3,4,5,6]

2nd largest:

5

Example 2

Input:

nums = [3,2,3,1,2,4,5,5,6]

k = 4

Output:

4

Sorted:



[1,2,2,3,3,4,5,5,6]

4th largest:

4

4. Constraints

- $1 \leq k \leq \text{nums.length} \leq 10^5$
- $-10^4 \leq \text{nums}[i] \leq 10^4$

5. Core Concept (Pattern / Topic)

Heap (Priority Queue)

Related Topics:

- Heap
- Quick Select
- Sorting

6. Thought Process (Step-by-Step Explanation)

Brute Force

Sort array.

Return:

`nums[n-k]`

Example:

[3,2,1,5,6,4]

Sorted:

[1,2,3,4,5,6]

2nd largest:

`nums[6-2]`

=

`nums[4]`

=

5

Complexity:

$O(n \log n)$

Better Observation

We don't need:



complete sorting

Need only:

kth largest

Min Heap Idea

Keep only:

k elements

inside heap.

Heap always stores:

k largest elements seen so far

The smallest element in heap becomes:

kth largest

Example

Input:

nums = [3,2,1,5,6,4]

k = 2

Insert 3

Heap:

[3]

Insert 2

Heap:

[2,3]

Size = k

Insert 1

Heap:

[1,3,2]

Size > k

Remove smallest:

1

Heap:

[2,3]

**Insert 5**

Heap:

[2,3,5]

Remove smallest:

2

Heap:

[3,5]

Insert 6

Heap:

[3,5,6]

Remove:

3

Heap:

[5,6]

Insert 4

Heap:

[4,6,5]

Remove:

4

Heap:

[5,6]

Answer:

5

7. Visual / Intuition Diagram (ASCII Diagram)

```
Input:
3 2 1 5 6 4
Keep only:
2 largest elements
Heap evolution:
[3]

[2, 3]

[2, 3]

[3, 5]

[5, 6]
```



```
[5,6]
Top:
5
Kth largest.
```

8. Pseudocode

```
Create min heap

for each number:

    push into heap

    if size > k:

        remove smallest

return heap top
```

9. Code Implementation

✓ Python

```
import heapq

class Solution:

    def findKthLargest(
        self,
        nums,
        k
    ):

        heap = []

        for num in nums:

            heapq.heappush(
                heap,
                num
            )

            if len(heap) > k:

                heapq.heappop(heap)

        return heap[0]
```

 Java

```
import java.util.*;

class Solution {

    public int findKthLargest(
        int[] nums,
        int k) {

        PriorityQueue<Integer> pq =
            new PriorityQueue<>();

        for (int num : nums) {

            pq.offer(num);

            if (pq.size() > k) {

                pq.poll();

            }

        }

        return pq.peek();

    }

}
```

10. Time & Space Complexity**Time Complexity**

Heap size:

k

Each insertion:

 $O(\log k)$

Total:

 $O(n \log k)$ **Space Complexity** $O(k)$ **11. Common Mistakes / Edge Cases****Mistake 1**

Using max heap of all elements.

Works but unnecessary.

**Mistake 2**

Returning heap size.

Need:

heap[0]

or

peek()

Mistake 3

Confusing kth largest with kth distinct.

Duplicates count.

12. Detailed Dry Run

Input:

nums = [3,2,3,1,2,4,5,5,6]

k = 4

Heap Evolution

Number	Heap
3	[3]
2	[2,3]
3	[2,3,3]
1	[1,2,3,3]
2	remove 1
4	remove 2
5	remove 2
5	remove 3
6	remove 3

Answer:

4

13. Common Use Cases

Ranking Systems



Find top performers.

Streaming Data

Maintain kth largest value continuously.

Analytics

Top-k statistics.

Search Engines

Top-k relevant results.

14. Common Traps

- Using sorting unnecessarily
- Confusing distinct values
- Using max heap incorrectly
- Returning wrong heap element

15. Builds To (Related Problems)

1. LC 703 — Kth Largest Element in a Stream
2. LC 347 — Top K Frequent Elements
3. LC 373 — K Smallest Pairs
4. LC 215 — Kth Largest Element in an Array
5. LC 295 — Find Median From Data Stream

16. Alternate Approaches

Approach	Time	Space	Notes
Sorting	$O(n \log n)$	$O(1)$	Simple
Min Heap	$O(n \log k)$	$O(k)$	Better
Quick Select	$O(n)$ Avg	$O(1)$	Optimal Interview Solution

17. Why This Solution Works

A min heap of size k always keeps the k largest elements encountered so far. Therefore, the smallest element inside that heap is exactly the kth largest element of the entire array.

18. Variations / Follow-Up Questions



Follow-Up 1

Find kth smallest element.

Use:

Max Heap of size k

or Quick Select.

Follow-Up 2

Find top-k largest elements.

Return all heap elements.

Follow-Up 3

Stream of incoming numbers.

→ LC 703 (Kth Largest Element in a Stream)